04-04. Replay Stage 구현 분석

개요

Replay Stage는 TVU 파이프라인의 마지막 단계로, Window Service에서 완성된 블록 데이터를 받아 트랜잭션을 재실행하고 상태를 업데이트하며 합의에 참여하는 핵심 컴포넌트입니다. 이 단계는 포크 선택, 투표 생성, 그리고 Tower BFT 합의 메커니즘을 구현합니다.

ReplayStage 구조체

```
pub struct ReplayStage {
t_replay: JoinHandle<()>, // 메인 재실행 스레드
commitment_service: AggregateCommitmentService, // 커밋먼트 집계 서비스
}
```

ReplayStage는 두 개의 주요 컴포넌트로 구성됩니다:

1. **Replay Thread**: 블록 재실행과 합의를 담당하는 메인 스레드
2. **Commitment Service**: 블록 커밋먼트를 집계하고 관리하는 서비스

ReplayStageConfig 설정

```
pub struct ReplayStageConfig {
pub vote_account: Pubkey, // 투표 계정
```

```
pub authorized_voter_keypairs: Arc<RwLock<Vec<Arc<Keypair>>>>, // 인증된 투표자 ㅋ
pub exit: Arc<AtomicBool>, // 종료 신호
pub rpc_subscriptions: Arc<RpcSubscriptions>, // RPC 구독 서비스
pub leader_schedule_cache: Arc<LeaderScheduleCache>, // 리더 스케줄 캐시
pub accounts_background_request_sender: AbsRequestSender, // 계정 백그라운드 요청
pub block_commitment_cache: Arc<RwLock<BlockCommitmentCache>>, // 블록 커밋먼트 ㅋ
pub transaction_status_sender: Option<TransactionStatusSender>, // 트랜잭션 상태
pub rewards_recorder_sender: Option<RewardsRecorderSender>, // 보상 기록 송신자
pub cache_block_meta_sender: Option<CacheBlockMetaSender>, // 블록 메타 캐시 송신
pub entry_notification_sender: Option<EntryNotifierSender>, // 엔트리 알림 송신자
pub bank_notification_sender: Option<BankNotificationSenderConfig>, // 뱅크 알림
pub wait_for_vote_to_start_leader: bool, // 리더 시작 전 투표 대기 여부
pub ancestor_hashes_replay_update_sender: AncestorHashesReplayUpdateSender, //
pub tower_storage: Arc<dyn TowerStorage>, // Tower 저장소
pub wait_to_vote_slot: Option<Slot>, // 투표 대기 슬롯
pub replay_slots_concurrently: bool, // 동시 슬롯 재실행 여부
}
```

메인 재실행 루프

초기화 과정

```
let (mut progress, mut heaviest_subtree_fork_choice) =
```

```
Self::initialize_progress_and_fork_choice_with_locked_bank_forks(
&bank_forks,
&my_pubkey,
&vote_account,
&blockstore,
);
```

ReplayStage는 시작 시 다음을 초기화합니다:

- **Progress Map**: 각 슬롯의 재실행 진행 상황 추적
- **Fork Choice**: 가장 무거운 서브트리 포크 선택 알고리즘
- **Tower**: 투표 기록 및 합의 상태
- **다양한 추적기들**: 중복 슬롯, 에포크 슬롯 등

스레드 풀 구성

```
// Thread pool to (maybe) replay multiple threads in parallel
let replay_mode = if replay_slots_concurrently {
let pool = rayon::ThreadPoolBuilder::new()
.num_threads(MAX_CONCURRENT_FORKS_TO_REPLAY)
.thread_name(|i| format!("solReplayFork{i:02}"))
.build()
.expect("new rayon threadpool");
ForkReplayMode::Parallel(pool)
} else {
```

```
ForkReplayMode::Serial

};



// Thread pool to replay multiple transactions within one block in parallel

let  replay_tx_thread_pool = rayon::ThreadPoolBuilder::new()

.num_threads(get_max_thread_count())

.thread_name(|i| format!("solReplayTx{i:02}"))

.build()

.expect("new rayon threadpool");
```

두 가지 병렬 처리 모드를 지원합니다:

1. **Fork 병렬 처리**: 최대 4개의 포크를 동시에 재실행
2. **Transaction 병렬 처리**: 블록 내 트랜잭션들을 병렬로 실행

핵심 처리 단계

1. 새로운 Bank Fork 생성

```
Self::generate_new_bank_forks(

&blockstore,

&bank_forks,

&leader_schedule_cache,

&rpc_subscriptions,

&mut  progress,
```

```
&mut replay_timing,
);
```

- Blockstore에서 새로운 완성된 슬롯 탐지
- 새로운 Bank 인스턴스 생성
- 부모-자식 관계 설정

2. 활성 Bank 재실행

```
let did_complete_bank = Self::replay_active_banks(
&blockstore,
&bank_forks,
&my_pubkey,
&vote_account,
&mut progress,
// ... 기타 매개변수들
);
```

활성 Bank들에 대해 다음 작업을 수행합니다:

- 트랜잭션 재실행
- 상태 업데이트
- 블록 완성 및 동결
- 포크 선택 알고리즘 업데이트

3. 중복 슬롯 처리

```
// Process cluster-agreed versions of duplicate slots

Self::process_ancestor_hashes_duplicate_slots(/* ... */);


// Check for any newly duplicate confirmed slots detected from gossip / replay

Self::process_duplicate_confirmed_slots(/* ... */);
```

다음과 같은 중복 상황을 처리합니다:

- 조상 해시 서비스에서 감지된 중복 슬롯
- Gossip이나 재실행에서 확인된 중복 슬롯

4. Gossip 투표 처리

```
Self::process_gossip_verified_vote_hashes(

&gossip_verified_vote_hash_receiver,

&mut  unfrozen_gossip_verified_vote_hashes,

&heaviest_subtree_fork_choice,

&mut  latest_validator_votes_for_frozen_banks,

);
```

Gossip 네트워크에서 수신된 검증된 투표를 처리하여 포크 선택에 반영합니다.

5. 포크 선택 및 투표

```
let SelectVoteAndResetForkResult {

vote_bank,
```

```
reset_bank,
heaviest_fork_failures,
} = Self::select_vote_and_reset_forks(
&my_pubkey,
&mut  tower,
&mut  heaviest_subtree_fork_choice,
&ancestors,
&descendants,
&progress,
&vote_tracker,
&cluster_slots,
&bank_forks,
&mut  voted_signatures,
&mut  has_new_vote_been_rooted,
);
```

Tower BFT 알고리즘을 사용하여:

- 투표할 최적의 포크 선택
- 필요시 포크 리셋
- 투표 생성 및 전송

포크 선택 알고리즘

HeaviestSubtreeForkChoice

```
enum HeaviestForkFailures {
LockedOut(u64), // 락아웃 상태
FailedThreshold(Slot, u64, u64, u64), // 임계값 실패
FailedSwitchThreshold(Slot, u64, u64), // 스위치 임계값 실패
NoPropagatedConfirmation(Slot, u64, u64), // 전파 확인 없음
}
```

포크 선택 시 다음 조건들을 검사합니다:

- **Lockout 검사**: 이전 투표로 인한 락아웃 상태 확인
- **Threshold 검사**: 충분한 스테이크 지지 확인
- **Switch Threshold**: 포크 전환을 위한 추가 임계값
- **Propagation**: 네트워크 전파 확인

Tower BFT 합의

```
pub  struct Tower {
node_pubkey: Pubkey, // 노드 공개키
threshold_depth: usize, // 임계값 깊이
threshold_size: f64, // 임계값 크기
vote_state_timestamps: Vec<Slot>, // 투표 상태 타임스탬프
last_vote: VoteTransaction, // 마지막 투표
last_timestamp: BlockTimestamp, // 마지막 타임스탬프
stray_restored_slot: Option<Slot>, // 복원된 이탈 슬롯
last_switch_threshold_check: Option<(Slot, Hash, u64)>, // 마지막 스위치 임계값 검사
```

```
}
```

Tower는 다음 기능을 제공합니다:

- **투표 기록 관리**: 과거 투표 이력 추적
- **Lockout 계산**: 투표로 인한 락아웃 기간 계산
- **슬래싱 방지**: 중복 투표 방지 메커니즘

성능 모니터링

ReplayLoopTiming

```
#[derive(Default)]
struct ReplayLoopTiming {
loop_count: u64, // 루프 실행 횟수
collect_frozen_banks_elapsed_us: u64, // 동결된 뱅크 수집 시간
compute_bank_stats_elapsed_us: u64, // 뱅크 통계 계산 시간
select_vote_and_reset_forks_elapsed_us: u64, // 투표 선택 및 포크 리셋 시간
start_leader_elapsed_us: u64, // 리더 시작 시간
reset_bank_elapsed_us: u64, // 뱅크 리셋 시간
voting_elapsed_us: u64, // 투표 시간
generate_vote_us: u64, // 투표 생성 시간
update_commitment_cache_us: u64, // 커밋먼트 캐시 업데이트 시간
replay_active_banks_elapsed_us: u64, // 활성 뱅크 재실행 시간
// ... 기타 타이밍 메트릭들
```

```
}
```

파티션 탐지

```
struct PartitionInfo {
partition_start_time: Option<Instant>,
}

impl PartitionInfo {
fn  update(
&mut  self,
partition_detected: bool,
heaviest_slot: Slot,
last_voted_slot: Slot,
reset_bank_slot: Slot,
heaviest_fork_failures: Vec<HeaviestForkFailures>,
)
}
```

네트워크 파티션을 탐지하고 복구를 모니터링합니다:

- 파티션 시작 시간 기록
- 파티션 해결 시간 측정
- 관련 메트릭 수집

투표 생성 및 전송

GenerateVoteTxResult

```
enum GenerateVoteTxResult {
NonVoting, // 비투표 검증자
Failed, // 생성 실패
Tx(Transaction), // 성공적으로 생성된 투표 트랜잭션
}
```

투표 생성 과정:

1. **투표 자격 확인**: 검증자가 투표 권한이 있는지 확인
2. **투표 트랜잭션 생성**: 선택된 포크에 대한 투표 트랜잭션 생성
3. **서명 및 전송**: 투표 트랜잭션에 서명하고 네트워크에 전송

리더 스케줄링

리더 시작 조건

```
fn  check_propagation_for_start_leader(
poh_slot: Slot,
parent_slot: Slot,
progress: &ProgressMap,
) -> bool
```

리더가 되기 위한 조건들:

- **전파 확인**: 이전 블록이 충분히 전파되었는지 확인
- **연속 리더십**: 연속된 리더 슬롯에 대한 특별 처리
- **투표 루팅**: 새로운 투표가 루트되었는지 확인

오류 처리 및 복구

Dead Slot 처리

```
fn mark_dead_slot(
blockstore: &Blockstore,
bank: &Bank,
root_slot: Slot,
err: &BlockstoreProcessorError,
// ... 기타 매개변수들
)
```

문제가 있는 슬롯을 처리합니다:

- **Dead 마킹**: 재실행 실패한 슬롯을 dead로 표시
- **포크 정리**: 관련 포크들을 포크 선택에서 제거
- **복구 시도**: 가능한 경우 슬롯 복구 시도

Duplicate Slot 복구

중복 슬롯이 탐지되면:

1. **클러스터 합의 확인**: 클러스터가 합의한 버전 확인

2. **로컬 데이터 정리**: 잘못된 로컬 데이터 제거

3. **올바른 데이터 요청**: Repair 서비스를 통해 올바른 데이터 요청

커밋먼트 관리

AggregateCommitmentService

```
let (lockouts_sender, commitment_service) = AggregateCommitmentService::new(
exit.clone(),
block_commitment_cache.clone(),
rpc_subscriptions.clone(),
);
```

커밋먼트 서비스는 다음을 관리합니다:

- **블록 커밋먼트**: 각 블록의 커밋먼트 레벨 추적
- **RPC 업데이트**: 클라이언트에게 커밋먼트 상태 알림
- **캐시 관리**: 커밋먼트 정보 캐싱

Replay Stage는 Solana의 합의 메커니즘의 핵심으로, 복잡한 포크 선택 알고리즘과 Tower BFT 합의를 통해 네트워크의 안전성과 활성을 보장합니다.